



T.C.
MARMARA
UNIVERSITY



FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

CSE4088 Introduction to Machine Learning
Term Project - Data Preprocessing Report

Project Title

Email Spam Detection

Group#7 Members

Emir Büçkün - 150119024

Mehmet Sina Çağlar - 150123821

Fatma Melisa Küçük - 150119916

İrem Kubalas - 150119721

Ayşenur Tüfekçi - 150119699

Instructor

Assoc. Prof. Murat Can Ganiz

Table of Contents

1. Data Cleaning Steps.....	3
1.1. Data Extraction and Loading.....	3
1.2. Cleaning the Labels.....	3
1.3. Handling Missing Data.....	3
1.4. Message Cleaning.....	3
1.5. Saving Processed Data.....	3
2. Exploratory Data Analysis (EDA).....	4
2.1. First Few Rows of the Dataset.....	4
2.2. Basic Information About the Cleaned Dataset.....	4
2.3. Message Length Statistics.....	5
2.4. Distribution of Classes.....	6
2.5. Relationship Between Message Length and Label.....	6
2.6. Top 10 Most Frequent Words In Cleaned Messages.....	7
3. Rationale for Feature Selection and Engineering.....	8
3.1. Engineered Features.....	8
3.2. Rationale for Feature Selection.....	9
3.2.1. Message Length and Word Count.....	9
3.2.2. Punctuation Count.....	9
3.2.3. Capital Letter Ratio.....	9
3.2.4. Spam Keyword Count.....	9

1. Data Cleaning Steps

1.1. Data Extraction and Loading

Email data in data/enron_spam.zip is extracted into the data/enron_data folder using the “zipfile” module. Folder paths are defined for ham and spam using the “os” module. The “load_messages” function that processes text files in a specified directory and appends their content, along with an associated label to a data structure named “data” is defined. Ham and spam messages are loaded using this function.

1.2. Cleaning the Labels

The categorical labels “ham” and “spam” are converted to numerical values (0 and 1 respectively) using map() function.

1.3. Handling Missing Data

The number of missing values per column is checked. Dropped rows with missing values to ensure data quality.

1.4. Message Cleaning

The “clean_message” function is defined. This function takes a string as input (text message in this case) and uses a regular expression to replace all characters in the string that are not letters(anything other than a-z or A-Z) with a space. It also converts all characters to lowercase and keeps only words that are not included in “stop_words”. This function is applied on the “Message” column.

1.5. Saving Processed Data

The cleaned data is saved to a new CSV file “cleaned_enron_dataset.csv” for future use.

2. Exploratory Data Analysis (EDA)

2.1. First Few Rows of the Dataset

Table 2.1.1 shows the first few rows of the dataset after cleaning, providing a snapshot of the dataset structure. Each row represents an email or message, and the dataset has been preprocessed to include a cleaned version of the messages (Clean_Message) and their corresponding labels (Label), where 0 represents "ham" and 1 represents "spam."

Index	Message	Clean_Message	Label
0	Subject: christmas tree farm pictures\n	subject christmas tree farm pictures	0
1	Subject: vastar resources , inc .\ngary , prod...	subject vastar resources inc gary production h...	0
2	Subject: calpine daily gas nomination\n- calpi...	subject calpine daily gas nomination calpine d...	0
3	Subject: re : issue\nfyi - see note below - al...	subject issue fyi see note already done stella..	0
4	Subject: meter 7268 nov allocation\nfyi .\n- -...	subject meter nov allocation fyi forwarded lau...	0

Table 2.1.1 First few rows of the dataset

2.2. Basic Information About the Cleaned Dataset

Table 2.2.1 summarizes the cleaned dataset. It contains 5,172 messages, all of which are complete (no missing values). The Clean_Message column is text data, while the Label column is numeric.

#	Column	Non-Null Count	Dtype
0	Message	5172	object
1	Clean_Message	5172	object
2	Label	5172	int64

Table 2.2.1 Basic information about the cleaned dataset

2.3. Message Length Statistics

Table 2.3.1 shows descriptive statistics of message lengths, measured by the number of characters. Most messages are relatively short, with a median length of 529 characters. The distribution, however, has a long tail, with some messages exceeding 31,000 characters.

Name	Message_Length
count	5172.00
mean	1030.15
std	1505.23
min	10.00
25%	238.00
50%	529.00
75%	1214.25
max	31860.00

able 2.3.1 Message length stats

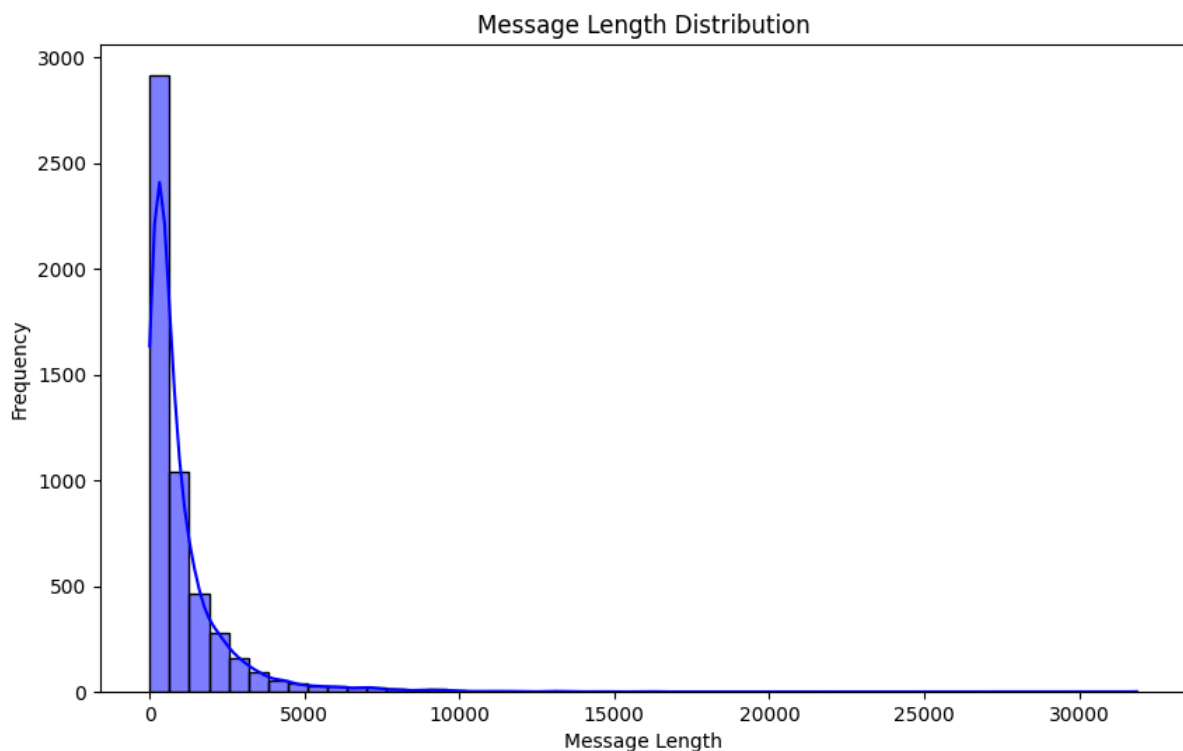


Fig 2.3.2 Message length distribution

2.4. Distribution of Classes

The dataset is imbalanced, with a significantly higher number of "ham" messages compared to "spam" messages.

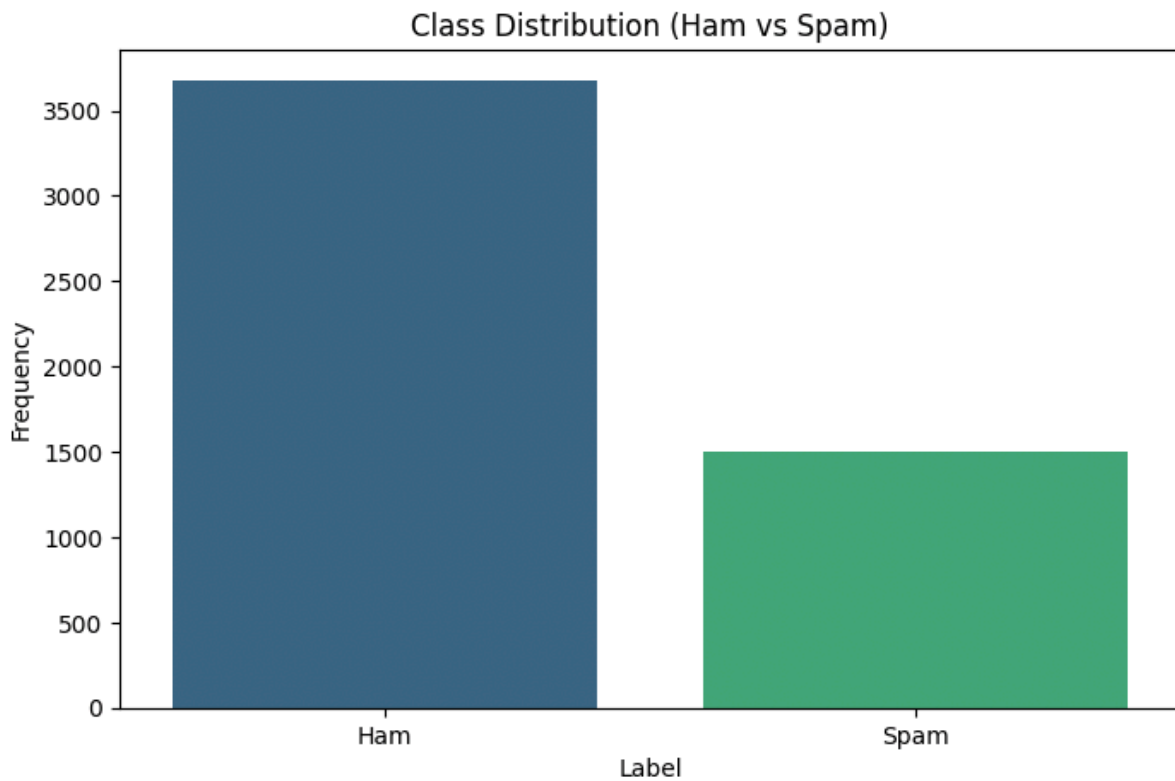


Fig 2.4.1 Class distribution of ham and spam labels

2.5. Relationship Between Message Length and Label

A boxplot in Fig 2.5.1 illustrates the relationship between message length and labels. Spam messages tend to be shorter, while ham messages have a wider range of lengths.

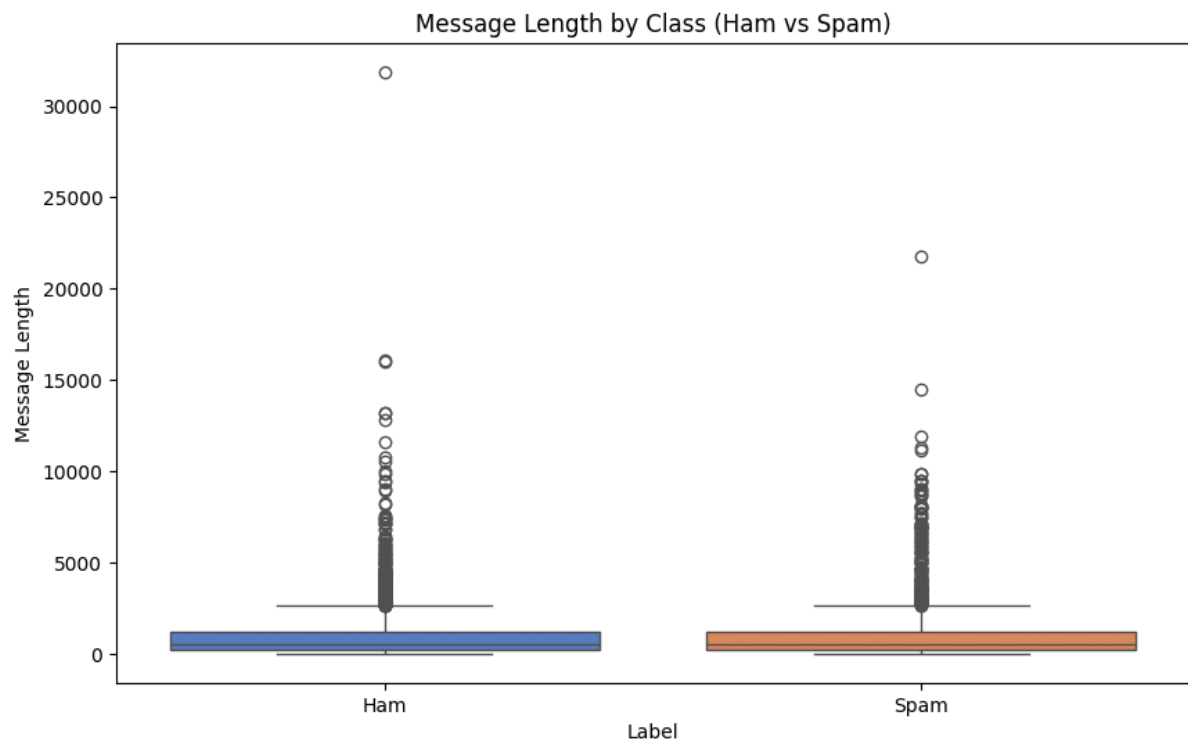


Fig 2.5.1 Boxplot of message length by class (ham vs spam)

2.6. Top 10 Most Frequent Words In Cleaned Messages

The most frequent words in the cleaned messages provide insights into the common themes of the dataset. Fig 2.6.1 visualizes these words, emphasizing their relative importance.

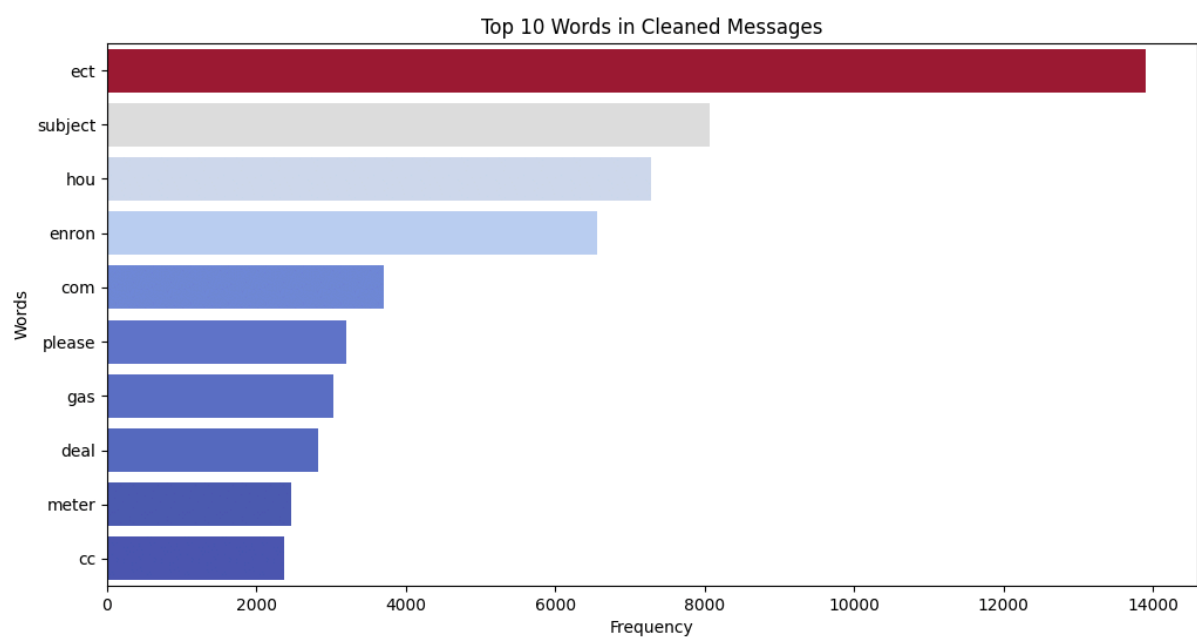


Fig 2.6.1 Top 10 words in cleaned dataset

3. Rationale for Feature Selection and Engineering

3.1. Engineered Features

Message_Length: Length of the message text. It is calculated by applying “`apply(len)`” on the Message column.

Word_Count: Number of words in the cleaned message. The text in `Clean_Message` is splitted into a list of words using “`x.split()`”. The words are counted with `len()`.

Punctuation_Count: Total punctuation marks in the original message. It is calculated by iterating over each character in the original message and checking if the character is in “`string.punctuation`”.

Capital_Letter_Ratio: Proportion of capital letters to total characters. It is calculated by iterating over each character in the original message and checking if the character is uppercase using “`char.isupper()`”.

Spam_Keyword_Count:

Identify Spam Keywords:

- TF-IDF (Term Frequency - Inverse Document Frequency) measures how important a word is in a collection of texts.
- `TfidfVectorizer` is initialized to extract the top 50 terms from the spam messages, ignoring common English stopwords.

Calculate Importance:

- `get_feature_names_out()` is used to retrieve the words selected by the vectorizer.
- TF-IDF scores of each term across all messages to determine their overall importance are summed.
- The words are combined with their corresponding scores using `zip()`.

Sort Terms By Importance:

- The words are sorted by their importance scores in descending order using `sorted()`.

Extract Top 10 Words:

- First 10 words in “spam_keywords_sorted” are stored in “spam_keywords”.

Count Calculation:

- Using the dynamically generated spam_keywords, the spam keywords appearing in each message are counted.

3.2. Rationale for Feature Selection

Each feature was chosen to capture distinct characteristics of the text while balancing interpretability and predictive power:

3.2.1. Message Length and Word Count

Measure the verbosity of a message, which may correlate with spam or non-spam behavior. Longer or overly brief messages can exhibit patterns indicative of spam.

3.2.2. Punctuation Count

Reflects stylistic patterns, such as excessive use of punctuation, which is common in spam messages for emphasis. (e.g. “Click now!!!”).

3.2.3. Capital Letter Ratio

Spam messages often contain a higher proportion of capital letters to grab attention.

3.2.4. Spam Keyword Count

Helps identify key phrases that are commonly found in spam messages.